MODULE 3 · LECTURE 18

CS 6530 Applied Cryptography

Public-Key Cryptography: From Secure Mathematics to Real-World Failures

Module 3 – Public-Key Cryptography

FROM CRYPTOGRAPHIC THEORY → DEPLOYED SOFTWARE

What Is a CVE — and What Can It Tell Us?

CVE = Common Vulnerabilities and Exposures

A CVE ID is a common identifier for a publicly disclosed cybersecurity vulnerability.

CVE-2024-31497

CVE	2024	31497
Common identifier	Year component	Sequence identifier

The year reflects when the CVE ID was assigned — not when the vulnerability first existed.

CVE / VULNERABILITY
CVE-2024-31497 (PuTTY ECDSA)
One specific product's vulnerability.

→

CRYPTOGRAPHIC FAILURE CLASS
🎲 **Nonce / Randomness**
What went wrong cryptographically.

A CVE identifies a vulnerability; the failure class explains why the cryptography became vulnerable.

One failure class can recur across multiple products and CVEs.

🎲 Randomness/Nonce	🔑 Key/Param Gen	✅ Validation/Verify	⏱️ Side Channel
📦 Encoding/Padding	🔄 Secret Reuse	🔻 Downgrade/Weak Para	💥 Robustness/DoS

INSIGHT

CVE tells us where a vulnerability was recorded; cryptographic analysis tells us what security property failed and why.

? THINK LIKE AN ENGINEER

Does an ECC-related CVE mean the elliptic-curve discrete-log problem itself has been broken?

ANSWER: NO

Not necessarily — it may lie in randomness, validation, implementation, protocol use, or side channels.

A CVE identifies a vulnerability; understanding its security significance requires identifying the underlying cryptographic failure.

REAL-WORLD RSA VULNERABILITIES

RSA in Practice — What Has Actually Failed?

🎲 🔑 RANDOMNESS / KEY GEN.

CVE-2017-15361 · ROCA · 2017
Weak Infineon RSA keygen
🔑 ***Secure RSA needs secure keygen.***

CVE-2008-0166 · Debian · 2008

Debian bug crippled key randomness

🎲 ***RSA can't fix weak randomness.***

📦 ✅ ENCODING/VERIF.

CVE-2026-50722 · Libreswan · 2026
AUTH didn't verify DER encoding

✅ ***Verify encoding exactly.***

CVE-2023-45853 · Marvin · 2023

Timing leak enabled oracle attack

📦 ⏱ ***RSA processing must not leak timing.***

⏱ SIDE CHANNEL

CVE-2023-50782 · 2023
python-cryptography: RSA decrypt timing leak

CVE-2017-7526 · Libgcrypt · 2017

RSA exponentiation leaked via cache timing

⏱ ***Secrets can leak without factoring n.***

INSIGHT

The mathematics may remain strong while randomness, key generation, encoding, verification or implementation behaviour fails.

? THINK LIKE AN ENGINEER

What was defeated in these examples — the hardness of factoring, or the system around RSA?

ANSWER: THE SYSTEM AROUND RSA

These failures arose from key creation, encoding/verification and side-channel behaviour — not a general break of RSA factorization.

Secure RSA requires secure randomness, key generation, encoding, verification and side-channel-resistant implementation.

REAL-WORLD KEY-AGREEMENT VULNERABILITIES

DH / ECDH in Practice — What Has Actually Failed?

DH and ECDH both give key agreement; a flaw in one doesn't affect the other.

✅ PEER-INPUT VALIDATION

ECDH · CVE-2026-26007 · 2026

No subgroup check on peer's EC key (S=dP).

ECDH · CVE-2018-5383 · Bluetooth · 2018

Weak EC public-key validation in pairing.

✅ ***Validate EC inputs; check subgroup rules.***

🔄🔑 SECRET/PARAM.

DH · CVE-2016-0701 · OpenSSL · 2016

Reused private DH exponent + non-safe parameters

🔄 ***Fresh private exponents + safe/validated parameters.***

DH · CVE-2015-4000 · Logjam · 2015

TLS downgrade to weak DH.

Strong groups, no downgrade.

⏱️ SIDE CHANNEL

DH · Raccoon Attack · 2020

Shared-secret timing could leak info; secret reuse raised exploitability.

⏱️ ***Named attack — see Additional Learning Material.***

INSIGHT

A hard discrete-logarithm problem is only one part of secure key agreement.

? THINK LIKE AN ENGINEER

What engineering requirements recur across these DH/ECDH failures?

ANSWER — RECURRING REQUIREMENTS

Strong parameters • validated peer inputs • fresh secrets • downgrade resistance • side-channel resistance

Secure key agreement requires more than DLP/ECDLP hardness: parameters, peer inputs, fresh secrets, protocol negotiation and implementation behaviour all matter.

REAL-WORLD ELLIPTIC-CURVE VULNERABILITIES

ECC in Practice — What Has Actually Failed?

ECDH validation examples already seen: CVE-2026-26007 · CVE-2018-5383

NONCE/RANDOM.

CVE-2026-22698 · RustCrypto SM2 · 2026
Used 32 bits, not 256 (unit mismatch).
32→256: nonce size matters.

CVE-2024-31497 · PuTTY · 2024
Biased ECDSA nonces for NIST P-521.
≈60 signatures → private-key recovery.

VERIFY/VALIDATE

CVE-2024-48949 · elliptic EdDSA · 2024
Verification skipped range-check on S.

CVE-2020-0601 · CurveBall · 2020
CryptoAPI mis-validated EC parameters.

Verify completely, not partially.

SIDE CHANNEL

CVE-2018-12438 · ROHNP · 2018
ECDSA signing cache leakage enabled key recovery.

Named attack — see Additional Material.

ROBUSTNESS

CVE-2022-0778 · OpenSSL · 2022
Malformed params hang BN_mod_sqrt().

Crypto input handling must fail safely.

Different CVEs — recurring failure classes.

INSIGHT
EC failures recur where math, implementation and untrusted input meet.

THINK LIKE AN ENGINEER

EC equations are correct. What other boundaries must still be protected?

ANSWER

Randomness • input validation • verification • side channels • robust errors

Secure elliptic-curve cryptography requires secure implementation of the complete operation — not merely correct curve arithmetic.

CROSS-CUTTING LESSONS

Different Algorithms — Recurring Failure Classes

RSA	DH / ECDH	ECC
Factoring hardness	*DLP / ECDLP hardness*	*EC discrete-log hardness*

↓ *converge on the same recurring failure classes* ↓

RANDOMNESS/KEYGEN

Predictable randomness.

Debian • ROCA • PuTTY

VALIDATION/VERIFICATION

Weak input validation.

Bluetooth • CurveBall

ENCODING/ROBUST INPUT

Weak encoding checks.

Libreswan • Marvin • OpenSSL

SIDE CHANNELS

Timing/cache leakage.

Libgcrypt • Raccoon • ROHNP

SECRET LIFECYCLE

Reused ephemeral secrets.

OpenSSL DH

PROTOCOL/PARAM.

Weak params, downgrade.

Logjam

Secure mathematics does NOT mean a secure cryptographic system.
Requires: Primitive + Parameters + Randomness + Validation + Implementation + Protocol Use

INSIGHT

Different hard problems, yet many deployed failures recur at the same boundaries.

? THINK LIKE AN ENGINEER

Did these failures require breaking the underlying hard problem at its intended security strength?

NO

Most exploited generation, validation, encoding, secret handling, implementation or protocol use. Logjam made discrete-log computation feasible by downgrading DH to weak parameters.

Choosing a mathematically strong primitive is necessary — but secure deployment requires protecting every boundary around it.

MODULE SYNTHESIS

Public-Key Cryptography — What Can It Give Us?

🔐 ENCRYPTION / KEY TRANSPORT

Protect a secret for its intended holder.

RSA-OAEP

$PK_B + M \rightarrow C$

$SK_B + C \rightarrow M$

Use OAEP randomized encoding — not textbook RSA.

🤝 KEY AGREEMENT

Derive a shared secret without transmitting it.

DH • ECDH • X25519

$Alice \Longrightarrow K_{shared} \Longleftarrow Bob$

$K_{shared} \rightarrow K_{session}$ (KDF)

Derives key material — does not itself encrypt data.

✍ DIGITAL SIGNATURE

Authenticate signed data and detect modification.

RSA-PSS • ECDSA • Ed25519

$SK_A + M \rightarrow$ Sig

$PK_A + M +$ Sig $\rightarrow$ Valid?

Signing is not encryption; verify correctly too.

Whichever function we choose, secure use still depends on:

Strong parameters • Secure randomness • Key protection • Correct validation • Side-channel resistance • Appropriate protocol use

INSIGHT

Algorithm selection should follow the required cryptographic function — not precede it.

? THINK LIKE AN ENGINEER

For a new secure application, should we begin by asking "RSA or ECC?"

NOT FIRST

First ask: What cryptographic function is required?

🔐 *Encryption/Key Transport* • 🤝 *Key Agreement* • ✍ *Digital Signature*

Public-key cryptography provides distinct tools for encryption/key transport, key agreement and digital signatures — each must be selected and used for its intended purpose.

THE QUESTION WE CANNOT YET FULLY ANSWER

You Have Bob's Public Key — But Is It Really Bob's?

Alice receives a key presented as Bob's: PK_B

WHAT ALICE CAN VERIFY

- Algorithm
- Params/Curve
- Encoding
- Valid Point/Group
- Operation Works

All check out — the key is mathematically well-formed.

EVE'S KEY LOOKS VALID TOO

Eve supplies PK_E, claiming it is Bob's.

- Valid
- Correctly Encoded
- Correct Curve
- Algorithm-Accepted

Yet it belongs to Eve — not Bob.

But who says this key belongs to Bob?

A VALID PUBLIC KEY IS NOT AN AUTHENTICATED PUBLIC KEY

Missing: a trustworthy way to bind an identity to a public key.

Still open: key storage • linking identity to a key • trust for validation • expiry/compromise

Next → Key Management • Certificates • PKI

INSIGHT

Cryptographic validity answers "Is this a valid key?" Authentication must answer "Whose key is it?"

THINK LIKE AN ENGINEER

ECDH gives Alice and Bob the same shared secret. What additional property is needed before Alice can say, "I shared it with Bob"?

AUTHENTICATION

Alice needs assurance that the public key used in the exchange actually belongs to Bob.

Public-key mathematics provides cryptographic operations; secure deployment also requires trustworthy key management and identity-to-key binding.

REFERENCES & FURTHER READING

Sources for Real-World Cryptographic Vulnerabilities

VULNERABILITY SOURCES

CVE Program
Common identifiers and records for publicly disclosed vulnerabilities.

NIST NVD
Vulnerability records, affected products and technical descriptions.

CERT/CC
Technical vulnerability notes and coordinated disclosure.

VENDOR ADVISORIES

OpenSSL
RSA, DH and elliptic-curve implementation vulnerabilities.

pyca/cryptography
Including EC subgroup-validation vulnerabilities.

Debian
Including the OpenSSL predictable-randomness incident.

PuTTY
Including the P-521 ECDSA nonce-bias vulnerability.

RESEARCH SOURCES

ROCA
Nemec et al., ACM CCS, 2017.

ROHNP
Ryan, TCHES, 2019.

Logjam
Adrian et al., ACM CCS, 2015.

Individual CVE identifiers are provided alongside the corresponding RSA, DH/ECDH and ECC cases in the preceding slides.

Named attacks including Bleichenbacher, Marvin, Logjam, Raccoon, ROCA, ROHNP, Minerva and CurveBall are explored in the accompanying Additional Learning Material.